

Notebook

February 8, 2026

1 DSA и ECDSA

Мы изучили, как проблема дискретного логарифма может быть использована для шифрования. DH и ECDH позволяют создать симметричный ключ при помощи асимметричной криптографии. К сожалению, их нельзя использовать для создания подписи. Но группы по модулю простого числа и группы точек эллиптических кривых можно использовать для создания подписи. Самые распространённые алгоритмы для этого - это DSA и его эллиптический вариант ECDSA.

1.1 Принцип работы

DSA нужна хеш-функция $Hash(x)$, мультипликативная группа Z_p^* по модулю простого числа p и генератор g порядка q , где q - наибольший простой делитель $p-1$ (т.е. g - генератор наибольшей подгруппы простого порядка группы Z_p^*). Мы уже это проходили, но чтобы найти такой генератор, надо выбрать случайный элемент $z \neq 1, p-1$ и вычислить $g = z^{\frac{p-1}{q}} \bmod p$. Очевидно, все эти параметры обычно стандартизированы.

Теперь, когда у нас есть $Hash(x)$, Z_p^* , g , q , мы можем создать ключевую пару. Ключи генерируются следующим образом:

1. Выбираем случайный $x \in (0, q)$ - это закрытый ключ
2. Вычисляем $y = g^x \bmod p$ - это открытый ключ

Теперь можно подписать сообщение m :

1. Выбираем случайный nonce (number used once, число, используемое один раз) $k \in (0, q)$
2. Вычисляем $r = (g^k \bmod p) \bmod q$. Если $r = 0$ идём на 1.
3. Вычисляем $s = \frac{Hash(m) + x \cdot r}{k} \bmod q$. Если $s = 0$ идём на 1.
4. Выводим (r, s) . Это подпись.

Для проверки подписи мы должны подтвердить, что $r, s \in (0, q)$ и проверить следующее равенство:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = r$$

Давайте проверим корректность:

$$(g^{\frac{Hash(m)}{s}} \cdot y^{\frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m)}{s}} \cdot g^{x \cdot \frac{r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m)}{s}} \cdot g^{x \cdot \frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m)}{s} + x \cdot \frac{r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m)}{s} + x \cdot \frac{r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m) + x \cdot r}{s}} \bmod p) \bmod q$$

$$(g^{\frac{Hash(m) + x \cdot r}{s}} \bmod p) \bmod q = (g^{\frac{Hash(m) + x \cdot r}{k}} \bmod p) \bmod q = (g^k \bmod p) \bmod q = r$$

В случае ECDSA у нас есть кривая $E(GF(p))$ и некоторая точка G в качестве генератора, порядок которой - простое число q , так что $q \cdot G = 0$ (Точка на Бесконечности). Нам также нужна хеш-функция $Hash(x)$. Для создания ключа мы снова:

1. Выбираем случайное $d \in (0, q)$ в качестве закрытого ключа
2. Вычисляем $Q = d \cdot G$, это наш открытый ключ

Алгоритм подписи сообщения m :

1. Выбираем случайный попсо $k \in (0, q)$
2. Вычисляем $(x_1, y_1) = k \cdot G$
3. Вычисляем $r = x_1 \bmod q$. Если $r = 0$, то возвращаемся на 1.
4. Вычисляем $s = \frac{Hash(m) + r \cdot d}{k} \bmod q$. Если $s = 0$, то возвращаемся на 1.
5. Выводим (r, s) в качестве подписи.

Проверка:

1. Проверяем, что $r, s \in (0, q)$
2. Вычисляем $(x_1, y_1) = \frac{Hash(m)}{s} \cdot G + \frac{r}{s} \cdot Q$
3. Проверяем $r = x_1 \bmod q$

Корректность:

$$\frac{Hash(m)}{s} \cdot G + \frac{r}{s} \cdot Q = \frac{Hash(m)}{s} \cdot G + d \frac{r}{s} \cdot G = \frac{Hash(m) + d \cdot r}{s} \cdot G = k \cdot G = (x_1, y_1)$$

1.2 Проблемы

Вы могли заметить, что оба примитива используют случайное значение k при подписи. Это основная проблема этого семейства алгоритмов, потому что **очень многое** может пойти не так. Имплементации DSA и ECDSA могут быть взломаны, если:

1. Используется предсказуемое k

2. Одинаковое k используется дважды
3. Используются k со статистическим отклонением (но это мы здесь не рассмотрим)

В первом случае, раз мы можем предсказать k , мы можем восстановить закрытый ключ:

1. Для DSA. $s = \frac{Hash(m) + x \cdot r}{k} \mod q \Rightarrow x = \frac{s \cdot k - Hash(m)}{r} \mod q$
2. Для ECDSA $s = \frac{Hash(m) + r \cdot d}{k} \mod q \Rightarrow d = \frac{s \cdot k - Hash(m)}{r} \mod q$

Во втором сценарии, поскольку одинаковые k были использованы для создания двух разных подписей:

$$\begin{aligned}
 k &= \frac{Hash(m) + x \cdot r}{s} \mod q \Rightarrow \\
 \frac{Hash(m_1) + x \cdot r_1}{s_1} &= \frac{Hash(m_2) + x \cdot r_2}{s_2} \mod q \Rightarrow \\
 (Hash(m_1) + x \cdot r_1)s_2 &= (Hash(m_2) + x \cdot r_2)s_1 \mod q \Rightarrow \\
 Hash(m_1)s_2 + x \cdot r_1 \cdot s_2 &= Hash(m_2)s_1 + x \cdot r_2 \cdot s_1 \mod q \Rightarrow \\
 Hash(m_1)s_2 - Hash(m_2)s_1 &= x(r_2 \cdot s_1 - r_1 \cdot s_2) \mod q \Rightarrow \\
 x &= \frac{Hash(m_1)s_2 - Hash(m_2)s_1}{r_2 \cdot s_1 - r_1 \cdot s_2} \mod q
 \end{aligned}$$

Т.е. мы снова можем вычислить x :

$$x = \frac{Hash(m_1)s_2 - Hash(m_2)s_1}{r_2 \cdot s_1 - r_1 \cdot s_2} \mod q$$

1.3 Задание

Поскольку атаки похожи, потренируемся только на DSA. Будет два подзадания. Сервер генерирует две ключевых пары. Он подписывает первое сообщение первой парой с использованием предсказуемого k :

```

class predictable_DSA(DSA):
    def get_k(self):
        return int(time.time())%(self.q-1)+1

```

Подсказка: сервер может находиться в другой временной зоне.

Потом он генерирует новое k другой функцией и использует его для создания второй и третьей подписей. Для них используется второй ключ. Ваша цель создать подпись сообщения 'give' при помощи первого ключа и сообщения 'flag' при помощи второго ключа, а потом отправить их на сервер. Удачи!

```
[ ]: import random
import time
from Crypto.Util.number import bytes_to_long
import hashlib
#Классы, используемые сервером
class DSA:
    def __init__(self,g,p,q):
        self.p=p
        self.g=g
        self.q=q
        self.x=random.randint(1,q-1)
        self.y=pow(g,self.x,self.p)

    def get_k(self):
        raise NotImplementedError()
    @staticmethod
    def message_to_hashnum(m):
        if not isinstance(m,bytes):
            raise TypeError()
        return bytes_to_long(hashlib.sha256(m).digest())
    def sign(self,m):
        r=0
        s=0
        while r==0 or s==0:
            k=0
            while k==0:
                k=self.get_k()
            r=pow(g,k,self.p)%self.q
            s=((DSA.message_to_hashnum(m)+(self.x*r)%self.q)%self.
↵q)*pow(k,q-2,self.q)%self.q
            return (r,s)
    def verify(self,m,r,s,otherY):
        if r==0 or s==0: return False
        w=pow(s,self.q-2,self.q)
        u1=(DSA.message_to_hashnum(m)*w)%self.q
        u2=(r*w)%self.q
        res=(pow(self.g,u1,self.p)*pow(otherY,u2,self.p)%p)%q
        return res==r

class predictable_DSA(DSA):
```

```

def get_k(self):
    return int(time.time())%(self.q-1)+1

class repeated_DSA(DSA):
    def get_k(self):
        global constant_k
        return constant_k

```

```

[ ]: import socket
import re
class VulnServerClient:
    def __init__(self, show=True):
        """Инициализация, подключаемся к серверу"""
        self.s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
        self.s.connect(('cryptotraining.zone',1350))

    def recv_until(self, symb=b'\n>'):
        """Получаем сообщения от сервера, по умолчанию до_
↪приглашения"""
        data=b''
        while True:

            data+=self.s.recv(1)
            if data[-len(symb):]==symb:
                break
        return data
    def getChallenge(self, show=True):
        data=self.recv_until()
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↪again.')
            return (None, None)
        if show:
            print (data)
        p=int(re.search(r'(?<=p=)\d+', data).group(0))
        q=(p-1)//2
        g=int(re.search(r'(?<=g=)\d+', data).group(0))
        y1=int(re.search(r'(?<=y1=)\d+', data).group(0))
        y2=int(re.search(r'(?<=y2=)\d+', data).group(0))
        mrs1=re.search(r'(?<=s1=)\(b'[a-z]+\',\d+,\d+)\', data).
        ↪group(0)[1:-1].split(',')
        m1,r1,s1=mrs1[0][2:-1].encode(),int(mrs1[1]),int(mrs1[2])
        mrs2=re.search(r'(?<=s2=)\(b'[a-z]+\',\d+,\d+)\', data).
        ↪group(0)[1:-1].split(',')
        m2,r2,s2=mrs2[0][2:-1].encode(),int(mrs2[1]),int(mrs2[2])

```

```

        mrs3=re.search(r'(?<=s3\)=)\(b\[a-z]+\',\d+,\d+)\',data).
↪group(0)[1:-1].split(',')
        m3,r3,s3=mrs3[0][2:-1].encode(),int(mrs3[1]),int(mrs3[2])
        #print (m1.encode(),r1,s1)
        return (p,q,g,y1,y2,m1,r1,s1,m2,r2,s2,m3,r3,s3)

    def checkSolution(self,rg,sg,rf,sf, show=True):
        """Проверка решения"""
        self.s.sendall((str(rg)+' '+str(sg)+' '+str(rf)+'_
↪'+str(sf)+'\n').encode())
        data=self.recv_until(b'\n')
        try:
            data=data.decode()
        except UnicodeDecodeError:
            print ('Error decoding unicode. Try connecting to server_
↪again.')
```

```

            return None
        if show:
            print (data)
        if data.find('flag')!=-1:
            return True
        else:
            data=self.recv_until(b'>')
            try:
                data=data.decode()
            except UnicodeDecodeError:
                print ('Error decoding unicode. Try connecting to_
↪server again.')
```

```

            return None
        if show:
            print (data)
        return False
    def __del__(self):
        self.s.close()

vs=VulnServerClient()
(p,q,g,y1,y2,m1,r1,s1,m2,r2,s2,m3,r3,s3)=vs.getChallenge()
```

```

[ ]: # You need to find x1 and x2
pDSA.x=x1
pDSA.y=y1
(r_g,s_g)=pDSA.sign(b'give')
pDSA.x=x2
pDSA.y=y2
(r_f,s_f)=pDSA.sign(b'flag')
```

```

[ ]: vs.checkSolution(r_g,s_g,r_f,s_f)
```

[]: